
STAGED METRIC-GATED GRPO FINE-TUNING PIPELINE FOR VISUAL NUMERIC REASONING

Kunal Ghosh

Purdue University
West Lafayette, Indiana, USA
ghosh178@purdue.edu
kg.aero@gmail.com

ABSTRACT

We study a staged metric-gated GRPO fine-tuning pipeline for visual numeric reasoning. The method combines a phase-wise curriculum over filtered MathVista subsets, a strict `<REASONING> / <SOLUTION>` response contract, metric-gated reward adaptation based on checkpoint evaluation, and alias-based checkpoint selection for non-monotonic training trajectories. We evaluate a rank-8 LoRA adaptation of `Qwen2.5-VL-7B-Instruct` under a fixed memory-constrained runtime profile. On the full `eval_overall_numeric` subset of MathVista `testmini` (456 examples), the best Phase B checkpoint improves exact match from 17.98% to 49.78%, tolerance accuracy from 18.20% to 50.66%, parseability from 25.44% to 97.81%, and composite score from 11.63% to 59.46% relative to the baseline adapter evaluated under the same decoding protocol. These results suggest that curriculum staging, metric-gated reward control, and checkpoint-aware continuation can substantially improve the stability of RL fine-tuning for multimodal numeric question answering.

Code: https://github.com/kgaoero/RL_GSPO_Qwen2.5_7B

1 INTRODUCTION

Reinforcement-learning fine-tuning for language and vision-language models is often summarized as a direct mapping from reward design to improved accuracy. In multimodal settings, however, the optimization problem is more complex (Yuan et al., 2025). Before correctness reward becomes informative, the policy must already emit outputs that are parseable, non-truncated, and structurally compatible with the evaluator (Shen et al., 2025). If those preconditions are unstable, RL updates can be difficult to interpret because an apparent correctness failure may in fact be a formatting or completion failure (Shen et al., 2025).

This paper investigates a staged metric-gated GRPO fine-tuning pipeline for visual numeric reasoning using `Qwen2.5-VL-7B-Instruct` as the base VLM. The pipeline leaves the GRPO objective unchanged and augments fine-tuning with filtered curriculum stages, a strict response contract, rule-based reward adaptation keyed to checkpoint metrics, and checkpoint aliases that allow continuation from the best structural or composite checkpoint rather than always from the most recent one. We analyze how this fine-tuning pipeline produces a structure-first and correctness-later trajectory for multimodal numeric question answering.

The implementation is designed to be auditable: controller states are saved, checkpoint aliases are explicit, reward weights are persisted, and evaluation artifacts contain both aggregate and sample-level records. This instrumentation is valuable in constrained-compute RL settings, where structural failures dominate early training and continuation decisions materially affect outcomes.

Figure 1 summarizes the loop. A base VLM is prepared with LoRA adapters, fine-tuned on a stage-filtered phase mixture, evaluated checkpoint by checkpoint, and then fed back into a controller that updates reward weights after each checkpoint evaluation. Table 1 shows that the curriculum is implemented as overlapping filtered views over the same MathVista split rather than as disjoint datasets.

Qwen2.5-VL Staged RL Pipeline

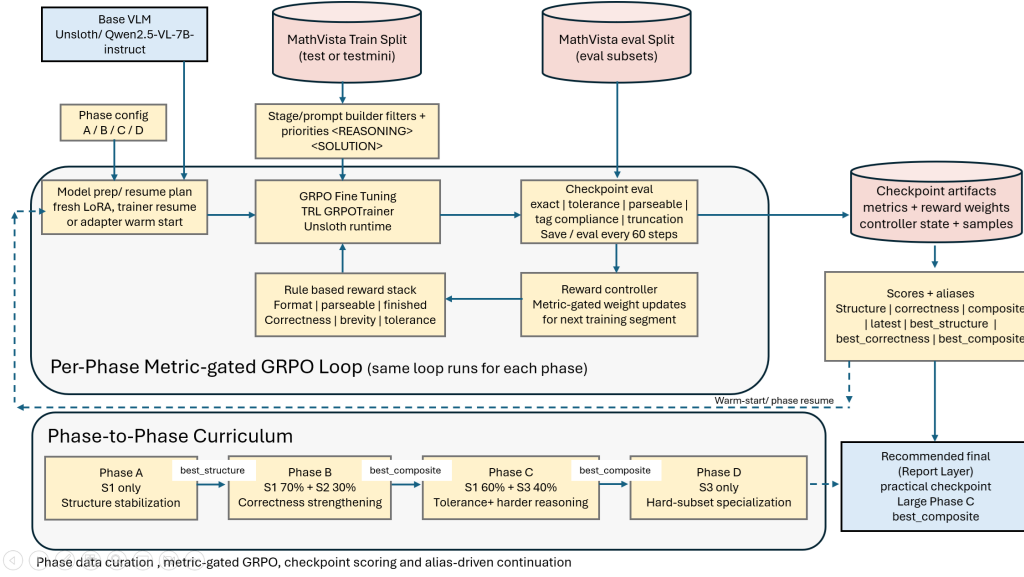


Figure 1: Overview of the staged metric-gated GRPO loop. The pipeline couples phase-specific data curation, GRPO fine-tuning, checkpoint-side evaluation, reward-weight adaptation, and alias-based continuation. The method studied here is the fine-tuning pipeline around GRPO rather than a new GRPO loss.

Contributions.

1. We document a staged GRPO fine-tuning pipeline for visual numeric reasoning that explicitly prioritizes structure stabilization before the main correctness gains.
2. We formalize the metric-gated reward controller and checkpoint aliasing logic so they can be read as an auditable algorithm rather than as scattered implementation details.
3. We report a controlled full-evaluation comparison across Baseline, Phase A, Phase B, and Phase C checkpoints on the same numeric `testmini` evaluation subset, with the highest primary score obtained by Phase B.

2 RELATED WORK

RL fine-tuning and preference optimization. Recent RLHF and preference-optimization work has primarily focused on new objectives or more efficient learning regimes. DeepSeekMath introduced GRPO as a group-relative alternative to PPO for mathematical reasoning (Shao et al., 2024). SPPO and Asynchronous RLHF studied, respectively, self-play preference optimization and faster off-policy online RLHF (Wu et al., 2025; Noukhovitch et al., 2025). More recent multimodal post-training studies have adapted GRPO- or RL-style optimization to vision-language reasoning through cold-start supervision, staged reinforcement learning, and explicit reasoning incentives (Huang et al., 2025; Wei et al., 2025; Chen et al., 2025). Our contribution is complementary: we leave the underlying GRPO objective unchanged and study how a staged fine-tuning pipeline combines GRPO with curriculum scheduling, metric-gated reward adaptation, and checkpoint-selection logic in a failure-prone multimodal setting.

Multimodal mathematical reasoning. MathVista highlights the difficulty of mathematical reasoning in visual contexts and serves as a benchmark for diagram, chart, table, and numeric question answering behavior (Lu et al., 2024). Recent work has addressed this problem with larger synthetic corpora, progressive curricula, staged reinforcement learning, or more explicit visual reasoning mechanisms (Zhang et al., 2025; Qi et al., 2025; Liu et al., 2025; Yuan et al., 2025). This paper focuses on the training-system side of the problem: a standard pretrained VLM paired with staged

curricula, metric-gated reward control, and checkpoint-aware continuation that prioritize structural reliability before stronger correctness pressure.

Curriculum and adaptive reward control. Our approach is closest in spirit to curriculum and adaptive-control approaches that treat fine-tuning as a sequence of regimes rather than as one stationary objective. Recent multimodal reasoning work has explored phased reinforcement learning, progressive curriculum RL, and rubric-based curriculum design as mechanisms for stabilizing difficult reasoning behavior (Liu et al., 2025; Yuan et al., 2025; Chen et al., 2026). The distinctive feature here is the granularity of adaptation: checkpoint metrics trigger reward changes directly within the fine-tuning pipeline.

3 PROBLEM SETUP AND DESIGN GOALS

We study a policy $\pi_\theta(c \mid x)$ that maps a visual question-answering input x to a completion c . Our method does not treat success as a single scalar notion of accuracy. Instead, it decomposes success into a sequence of preconditions:

1. the model must emit the required tagged structure,
2. the final answer must be parseable,
3. the answer must finish within the completion budget, and
4. the parsed answer must be correct under exact or tolerance-based matching.

The central question is therefore not only how to optimize correctness, but also when correctness reward becomes informative enough to emphasize. If structure is unstable, correctness reward is sparse because malformed or truncated outputs never reach a meaningful matching stage.

The implementation makes three concrete design choices.

- **Increasing-difficulty data staging.** The policy first sees easier integer-valued numeric problems before harder float and geometry-heavy subsets are emphasized.
- **Metric-gated rewards.** Reward weights change only after checkpoint evaluation, using measured failure modes rather than a fixed epoch schedule.
- **Alias-based continuation.** Checkpoint continuation is based on structure, correctness, or composite aliases rather than assuming the latest checkpoint is best.

Taken together, these choices define the central design logic of the method.

4 METHOD

4.1 TASK, RESPONSE CONTRACT, AND PHASE STRUCTURE

The code targets MathVista-style visual question answering in which the answer is primarily numeric and free-form. Each prompt is wrapped with a strict output contract:

```
<REASONING> ... </REASONING>
<SOLUTION> single numeric answer </SOLUTION>
```

This contract is important because both the reward functions and the evaluator depend on extracting a unique solution string from the completion. The pipeline also contains a separate multi-choice scaffold, but that branch is disabled by default and is not part of the main evidence.

The curriculum is organized into stages and phases. Stages are filtered subsets over the same dataset. In the configuration used for the main training path:

- **Stage 1** restricts to easier English free-form integer questions on synthetic scenes, tables, and natural images.
- **Stage 2** broadens to integer and float answers, adding tables, charts, plots, scientific figures, and natural images.

Filter	S1 Easy Numeric	S2 Medium / Float	S3 Hard Numeric
Stage ID	stage1.easy_numeric	stage2.float_numeric	stage3.hard_numeric
Goal	easy numeric stabilization	moderate precision and broader numeric coverage	harder multistep numeric reasoning
Question type	free_form	free_form	free_form
Answer type	integer	integer, float	integer, float
Language	english	english	english
Answer mode	numeric_free_form	numeric_free_form	numeric_free_form
Context families	synthetic scene	table	geometry diagram
	table	chart	plot
	natural image	plot	scientific figure
		scientific figure	abstract scene
Skills (any)	arithmetic reasoning	arithmetic reasoning	geometry reasoning
	statistical reasoning	statistical reasoning	algebraic reasoning
	numeric commonsense	scientific reasoning	scientific reasoning
		numeric commonsense	arithmetic reasoning
			statistical reasoning
Hard only	False	False	True

Table 1: Code-derived filter definitions for the three numeric curriculum stages used in the main training path. The stages overlap and are implemented as filtered views over the same MathVista split, not as fixed disjoint datasets.

- **Stage 3** emphasizes harder geometry, scientific-figure, plot, and abstract-scene problems.

Phases then mix these stages with different reward emphases:

- **Phase A:** Stage 1 only, structure stabilization.
- **Phase B:** 0.7 Stage 1 + 0.3 Stage 2, early correctness strengthening.
- **Phase C:** 0.6 Stage 2 + 0.4 Stage 3, harder reasoning and tolerance reward.

4.2 REWARD DECOMPOSITION

For a completion c , the method computes six per-sample reward components:

$$R(c) = \sum_{j=1}^6 w_j r_j(c), \quad (1)$$

where the components correspond to `format`, `parseable`, `finished`, `correctness`, `brevity`, and `tolerance`. The initial weights are phase-specific. Phase A places relatively high weight on structure and completion; Phase C raises correctness and activates tolerance reward.

The implementation never allows structure rewards to decay fully to zero. The guiding design principle is that, even when exact match becomes the main target, structural validity remains a necessary condition for receiving informative reward. The exact code-aligned reward definitions are reproduced in the supplement.

4.3 METRIC-GATED REWARD CONTROL

The reward controller updates weights after each checkpoint evaluation using explicit rules derived from observed metrics. Let p denote parseable-answer rate, s solution-tag compliance, r reasoning-tag compliance, m malformed rate, t truncation rate, and e exact match. The controller applies the following guards:

$$p < 0.85 \Rightarrow w_{\text{parseable}} \leftarrow \max(w_{\text{parseable}}, 0.75), \quad (2)$$

$$(s < 0.90) \vee (r < 0.88) \vee (m > 0.10) \Rightarrow w_{\text{format}} \leftarrow \max(w_{\text{format}}, 0.75), \quad (3)$$

$$(t > 0.15) \vee (\bar{\ell} > 0.85L) \Rightarrow w_{\text{finished}} \leftarrow w_{\text{finished}} + 0.25, \quad (4)$$

where $\bar{\ell}$ is average completion length and L is the completion budget.

Setting	Value	Used for
LoRA rank / alpha / max rank	8 / 8 / 8	all reported adapters and final evaluations
Model loading	4-bit, max sequence length 1280, image size 336	memory-constrained VLM loading
Training generation budget	max prompt length 320, max completion length 64	Phase A–C GRPO fine-tuning
Training sampling	two generations per prompt	GRPO group-relative updates
Final evaluation sampling	one completion per prompt	primary comparison protocol except for generation sampling randomness
Final evaluation cap	no subset cap	full selected evaluation subsets
Primary subset	eval_overall_numeric, 456 examples	main Baseline/A/B/C comparison

Table 2: Runtime and evaluation settings used for the reported final comparison. The memory-constrained hardware profile contains a small evaluation cap by default, but the final evaluation explicitly clears that cap and evaluates the full selected subsets.

Correctness is escalated only when structure is already stable. Stability requires

$$p \geq 0.92, \quad s \geq 0.95, \quad r \geq 0.93, \quad m \leq 0.05, \quad t \leq 0.08, \quad (5)$$

for a two-checkpoint window. If that condition holds and exact-match improvement over the window is below 0.02, the controller increases correctness weight by 0.50. All weights are then clamped to configured bounds. The rule set is intentionally simple enough to audit directly in saved checkpoint artifacts.

4.4 CHECKPOINT-SIDE EVALUATION AND ALIAS-BASED SELECTION

Instead of treating the most recent checkpoint as the default continuation point, the pipeline evaluates saved checkpoints and computes three aggregate scores:

$$S_{\text{struct}} = 0.35 p + 0.20 s + 0.20 r - 0.15 m - 0.10 t, \quad (6)$$

$$S_{\text{corr}} = 0.70 e + 0.20 a_{\text{tol}} + 0.10 p, \quad (7)$$

$$S_{\text{comp}} = 0.45 e + 0.15 a_{\text{tol}} + 0.15 p + 0.10 s + 0.05 r - 0.05 m - 0.05 t, \quad (8)$$

where a_{tol} is tolerance accuracy. These scores back four aliases: `latest`, `best_structure`, `best_correctness`, and `best_composite`. Cross-phase continuation typically resumes from `best_structure` or `best_composite`, while same-phase restarts may use full trainer resume from `latest`. This distinction is important in the implementation because the code supports both full trainer-state restoration and LoRA-only warm start.

5 EXPERIMENTAL SETTING AND EVALUATION PROTOCOL

Dataset and model. We use the Hugging Face release of MathVista, AI4Math/MathVista (AI4Math, 2026; Lu et al., 2024), and load unsloth/Qwen2.5-VL-7B-Instruct as the base VLM (Unsloth, 2026; Bai et al., 2025).

Training protocol. The reported training trajectory uses the MathVista `test` split for Phase A–C fine-tuning. Phase A trains on Stage 1, Phase B trains on the configured Stage 1/2 mixture, and Phase C trains on the configured Stage 2/3 mixture. The selected checkpoints for final evaluation are `phase_a/checkpoint-237`, `phase_b/checkpoint-180`, and `phase_c/checkpoint-240`.

Runtime profile. The main reported runs use the memory-constrained `kaggle_t4` profile: 4-bit loading, LoRA rank 8, max sequence length 1280, image size 336, gradient accumulation 4, two sampled generations per prompt during RL fine-tuning, max prompt length 320, and max completion length 64. The trainer uses TRL’s `GRPOTrainer` with sequence-level importance sampling and `dr_grpo` loss.

Table 3: Final full evaluation on `eval_overall_numeric` from MathVista `testmini` (456 examples).

Phase	Checkpoint	Exact	Tol	Parse	AvgR	Struct	Corr	Comp
Baseline	<code>baseline_rank8</code>	0.1798	0.1820	0.2544	-0.0880	0.0285	0.1877	0.1163
Phase A	<code>checkpoint-237</code>	0.4803	0.4868	0.9890	6.3857	0.7421	0.5325	0.5859
Phase B	<code>checkpoint-180</code>	0.4978	0.5066	0.9781	5.8754	0.7371	0.5476	0.5946
Phase C	<code>checkpoint-240</code>	0.4737	0.4846	0.9890	6.0361	0.7436	0.5274	0.5833

Table 4: Baseline versus the highest-scoring primary checkpoint, Phase B `checkpoint-180`.

Metric	Baseline	Phase B	Absolute Gain
Exact match	17.98%	49.78%	+31.80 pp
Tolerance accuracy	18.20%	50.66%	+32.46 pp
Parseable rate	25.44%	97.81%	+72.37 pp
Average reward	-0.0880	5.8754	+5.9634
Structure score	2.85%	73.71%	+70.86 pp
Correctness score	18.77%	54.76%	+35.99 pp
Composite score	11.63%	59.46%	+47.83 pp

Final evaluation protocol. The primary comparison evaluates Baseline, Phase A, Phase B, and Phase C on the same MathVista `testmini` subset, `eval_overall_numeric`. The final evaluation uses one sampled completion per prompt, max completion length 64, and no cap on the number of examples per subset. Stage-wise diagnostic evaluations are also computed on `stage1_easy_numeric`, `stage2_float_numeric`, and `stage3_hard_numeric`, but those diagnostics are not the primary metric.

Reproducibility. The implementation saves metrics, subset metrics, reward weights, controller state, controller decision logs, prompt-level records, and sample-level JSONL traces. The final evaluation artifacts include the primary-result CSV, stage-wise diagnostic CSV, subset-size JSON, and full JSON summary used to construct the tables below.

6 RESULTS

6.1 FULL PRIMARY EVALUATION

Table 3 reports the primary controlled comparison. The baseline adapter is evaluated with the same model runtime, LoRA shape, prompt contract, decoding budget, split, and evaluation subset as the trained adapters. Phase A produces the largest structural transition: parseability rises from 25.44% to 98.90% and composite score rises from 11.63% to 58.59%. Phase B obtains the highest final exact match, tolerance accuracy, correctness score, and composite score. Phase C remains above baseline but does not improve over Phase B on the primary evaluation subset.

6.2 IMPROVEMENT OVER BASELINE

Table 4 summarizes the highest-scoring primary checkpoint against the baseline. We report percentage-point gains rather than only relative gains because the baseline scores are low and relative changes can overstate the practical interpretation. The main result is a 31.80 percentage-point exact-match gain and a 32.46 percentage-point tolerance-accuracy gain at Phase B.

6.3 REWARD TRAJECTORY DURING TRAINING

Figure 2 shows the mean training reward logged by the trainer across the selected Phase A, Phase B, and Phase C runs. The phase boundaries are aligned by cumulative training step. Because each phase uses different reward weights, the plot should be interpreted as a within-pipeline training diagnostic rather than as a direct evaluation metric. The main pattern is that Phase A rapidly reaches high

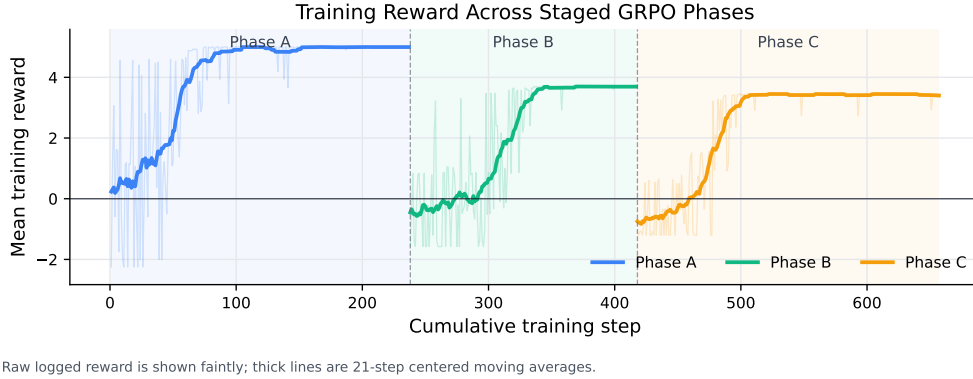


Figure 2: Training reward across the selected Phase A–C runs. Faint lines show raw per-step logged reward values from `trainer_state.json`; thick lines show 21-step centered moving averages.

Table 5: Stage-wise diagnostic evaluation on full `testmini` stage filters. These diagnostics are separate from the primary `eval_overall_numeric` comparison.

Phase	Eval subset	Exact	Tol	Parse	Trunc
Baseline	Stage 1	0.1641	0.1641	0.2872	0.6923
Baseline	Stage 2	0.1220	0.1220	0.1672	0.8293
Baseline	Stage 3	0.0924	0.0924	0.1681	0.8151
Phase A	Stage 1	0.4615	0.4615	0.9897	0.0051
Phase A	Stage 2	0.4321	0.4460	0.9895	0.0000
Phase A	Stage 3	0.3950	0.4034	0.9832	0.0084
Phase B	Stage 1	0.4718	0.4718	0.9846	0.0000
Phase B	Stage 2	0.4774	0.4983	0.9721	0.0035
Phase B	Stage 3	0.3866	0.4118	0.9664	0.0000
Phase C	Stage 1	0.4564	0.4564	0.9692	0.0000
Phase C	Stage 2	0.4321	0.4530	0.9861	0.0000
Phase C	Stage 3	0.4202	0.4286	0.9832	0.0000

structure-oriented reward, whereas Phases B and C operate under larger correctness pressure and therefore have lower absolute logged reward despite stronger downstream evaluation metrics.

6.4 STAGE-WISE DIAGNOSTICS

The primary evaluation subset is identical for all four targets. Stage-wise subsets are used as diagnostics to understand where each checkpoint gains or loses behavior. Table 5 shows that all trained phases improve over baseline on every diagnostic subset. Phase B obtains the highest Stage 1 and Stage 2 exact match, while Phase C obtains the highest Stage 3 exact match. This pattern is consistent with the curriculum: Phase C emphasizes the Stage 2/3 mixture, but that specialization does not translate into the highest aggregate primary score.

6.5 STRUCTURE IMPROVES BEFORE CORRECTNESS

The full evaluation supports the intended structure-first interpretation. The baseline produces many unparseable and truncated outputs, which makes correctness reward difficult to interpret. Phase A largely mitigates this failure mode: parseability becomes nearly saturated and average reward becomes strongly positive. Phase B then provides the highest correctness and composite scores. Phase C improves the hardest diagnostic subset but slightly reduces the aggregate primary metrics relative to Phase B.

This behavior is consistent with the controller design. Early reward pressure gives the model a stable response format and parseable final answer. Once that behavior is reliable, correctness and tolerance rewards become more informative. The non-monotonic Phase B to Phase C change also motivates alias-based checkpoint selection: later continuation is not guaranteed to dominate an earlier selected checkpoint on every evaluation view.

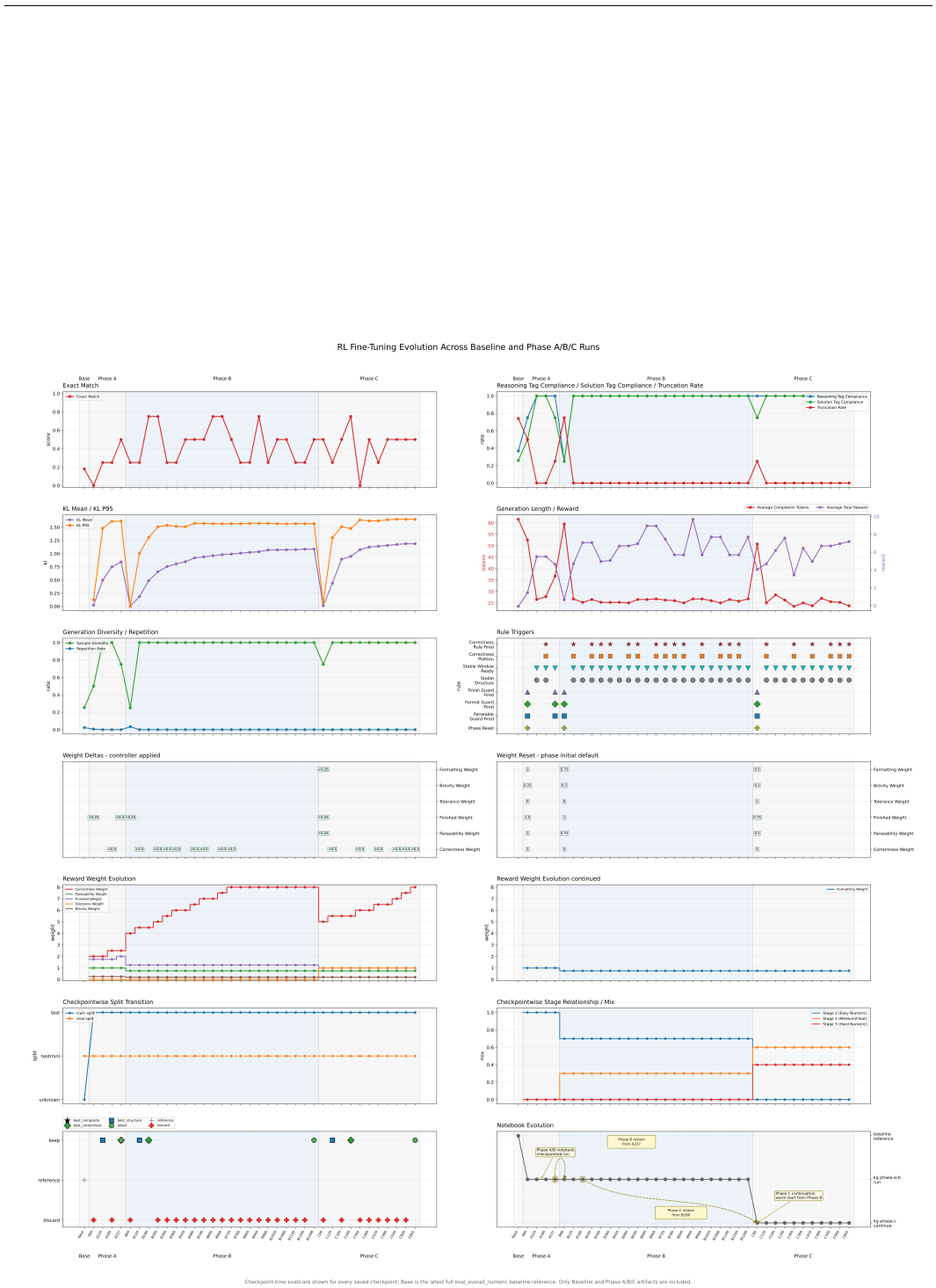


Figure 3: Training evolution across checkpoint-side milestones. The reported primary result uses the full evaluation in Table 3; this figure is retained as a qualitative view of the structure-first, correctness-later trajectory observed during training.

7 LIMITATIONS AND REPRODUCIBILITY

- The final primary evaluation uses the full numeric `testmini` subset, but it is still a single evaluation split with one sampled completion per prompt.
- The reported runs use one base model family, one LoRA rank, one memory-constrained runtime profile, and no seed sweep.
- Training uses the MathVista `test` split and evaluation uses `testmini`; broader held-out evaluation would be needed before making benchmark-level generalization claims.
- Checkpoint selection is guided by checkpoint-side diagnostics, so future work should separate checkpoint selection and final reporting with an additional held-out validation split when compute permits.
- The multi-choice branch and optional robustness stage have preliminary implementation support but are not part of the main evidence.

The study remains comparatively reproducible. It logs controller decisions at evaluated checkpoints, persists multiple checkpoint-selection aliases, saves sample-level traces, and includes unit tests for the most failure-prone logic. The supplement expands this record with explicit reward equations, evaluation formulas, controller thresholds, runtime settings, and transfer semantics.

8 CONCLUSION

We presented a staged metric-gated GRPO fine-tuning pipeline for visual numeric reasoning in a resource-constrained setting. The final full evaluation shows a substantial improvement over the baseline adapter: Phase B raises exact match from 17.98% to 49.78%, tolerance accuracy from 18.20% to 50.66%, parseability from 25.44% to 97.81%, and composite score from 11.63% to 59.46% on the same 456-example numeric `testmini` subset. The trajectory is structure-first and correctness-later: Phase A stabilizes output validity, Phase B gives the highest primary correctness, and Phase C improves the hardest diagnostic subset without improving the aggregate primary score. These findings support staged curriculum-based RL fine-tuning pipelines as a practical direction for multimodal numeric reasoning under constrained compute.

REFERENCES

- AI4Math. MathVista dataset card, 2026. URL <https://huggingface.co/datasets/AI4Math/MathVista>. Accessed: 2026-04-03.
- Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, Junyang Lin, et al. Qwen2.5-vl technical report, 2025. URL <https://arxiv.org/abs/2502.13923>.
- Shuang Chen, Yue Guo, Zhaochen Su, Yafu Li, Yulun Wu, Jiacheng Chen, Jiayu Chen, Weijie Wang, Xiaoye Qu, and Yu Cheng. Advancing multimodal reasoning: From optimized cold start to staged reinforcement learning, 2025. URL <https://arxiv.org/abs/2506.04207>.
- Yukun Chen, Jiaming Li, Longze Chen, Ze Gong, Jingpeng Li, Zhen Qin, Hengyu Chang, Ancheng Xu, Zhihao Yang, Hamid Alinejad-Rokny, Qiang Qu, Bo Zheng, and Min Yang. Rucl: Stratified rubric-based curriculum learning for multimodal large language model reasoning, 2026. URL <https://arxiv.org/abs/2602.21628>.
- Wenxuan Huang, Bohan Jia, Zijie Zhai, Shaosheng Cao, Zheyu Ye, Fei Zhao, Yao Hu, and Shaohui Lin. Vision-rl: Incentivizing reasoning capability in multimodal large language models, 2025. URL <https://arxiv.org/abs/2503.06749>.
- Zeyu Liu, Yuhang Liu, Guanghao Zhu, Congkai Xie, Zhen Li, Jianbo Yuan, Xinyao Wang, Qing Li, Shing-Chi Cheung, Shengyu Zhang, Fei Wu, and Hongxia Yang. Infi-mmr: Curriculum-based unlocking multimodal reasoning via phased reinforcement learning in multimodal small language models, 2025. URL <https://arxiv.org/abs/2505.23091>.

-
- Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2310.02255>.
- Michael Noukhovitch, Shengyi Huang, Sophie Xhonneux, Arian Hosseini, Rishabh Agarwal, and Aaron Courville. Asynchronous rlhf: Faster and more efficient off-policy rl for language models. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2410.18252>.
- Ji Qi, Ming Ding, Weihang Wang, Yushi Bai, Qingsong Lv, Wenyi Hong, Bin Xu, Lei Hou, Juanzi Li, Yuxiao Dong, and Jie Tang. Cogcom: A visual language model with chain-of-manipulations reasoning. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2402.04236>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Chuming Shen, Wei Wei, Xiaoye Qu, and Yu Cheng. Satori-rl: Incentivizing multimodal reasoning through explicit visual anchoring, 2025. URL <https://arxiv.org/abs/2505.19094>.
- Unsloth. Qwen2.5-vl-7b-instruct model card, 2026. URL <https://huggingface.co/unsloth/Qwen2.5-VL-7B-Instruct>. Accessed: 2026-04-03.
- Lai Wei, Yuting Li, Kaipeng Zheng, Chen Wang, Yue Wang, Linghe Kong, Lichao Sun, and Weiran Huang. Advancing multimodal reasoning via reinforcement learning with cold start, 2025. URL <https://arxiv.org/abs/2505.22334>.
- Yue Wu, Zhiqing Sun, Huizhuo Yuan, Kaixuan Ji, Yiming Yang, and Quanquan Gu. Self-play preference optimization for language model alignment. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2405.00675>.
- Ruifeng Yuan, Chenghao Xiao, Sicong Leng, Jianyu Wang, Long Li, Weiwen Xu, Hou Pong Chan, Deli Zhao, Tingyang Xu, Zhongyu Wei, Hao Zhang, and Yu Rong. V1-cogito: Progressive curriculum reinforcement learning for advanced multimodal reasoning, 2025. URL <https://arxiv.org/abs/2507.22607>.
- Renrui Zhang, Xinyu Wei, Dongzhi Jiang, Ziyu Guo, Yichi Zhang, Chengzhuo Tong, Jiaming Liu, Aojun Zhou, Shanghang Zhang, Peng Gao, and Hongsheng Li. Mavis: Mathematical visual instruction tuning with an automatic data engine. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2407.08739>.